Safe Proximal Policy Optimization for Sparse-Reward Classic Control: A PPO-Lagrangian Study on `MountainCar`

Hadis Amin (u8050066) Muhammad Rafif (u7964962)
Edwar Budiman (u7898974) Mukhit Onalbekov (u7812910)

Abstract

We study constrained reinforcement learning in the sparse-reward control task `MountainCar-v0`, building three agents of increasing capability: a Deep Q-Network baseline, a Proximal Policy Optimization (PPO) baseline, and *Safe PPO*, a dual-critic PPO-Lagrangian agent that respects a soft proportional speed constraint. The same energy-based reward shaping is used throughout, so the comparison isolates the safety mechanism. Safe PPO adds a second critic for the cost-value function and adapts a Lagrangian multiplier λ by dual ascent against a cost budget. A two-stage study first screens six PPO bases by a stability-aware score, then sweeps the safety hyperparameters on the top three, giving 18 constrained configurations over three seeds. The best Safe PPO matches the unconstrained PPO return (47.3 ± 0.3 against 47.8) while meeting the budget. Our central finding is that *base selection decides feasibility*: a high-cost base fails the constraint in every safety configuration, whereas a low-cost base of equal return yields the feasible winner at no measurable return cost. The multiplier λ tracks the violation magnitude, acting as an interpretable, automatic safety regulator. We release all code, configurations, training-process videos, and analysis scripts.

1 Introduction

Reinforcement learning (RL) has produced impressive empirical results across games and control [5, 11], but standard RL implicitly assumes that the agent is free to maximize reward irrespective of side effects. Many practical applications — robotics, autonomous driving, healthcare — require the agent to satisfy additional safety or operational constraints. Constrained Markov Decision Processes (CMDPs) [2] formalize this setting by decorating each transition with a scalar *cost* signal, and asking the agent to maximize return subject to a constraint on expected cumulative cost.

Despite a rich body of constrained-RL work [1, 4, 9, 12, 13], the classic control benchmark `MountainCar-v0` [3, 7] is rarely used to study safety, because its dynamics are too simple and its reward is too sparse to make exploration interesting. We argue that exactly these properties make it a useful *didactic* testbed: the algorithmic interactions between sparse reward, reward shaping, and constraint enforcement become visible without confounding effects from large neural networks or visual inputs.

Contributions.

1. We assemble three progressively complex agents — DQN, PPO, and Safe PPO — on a shared, energy-shaped `MountainCar` environment, with identical hyperparameters where possible, to isolate the contribution of the safety mechanism.
2. We introduce a soft, *proportional* speed cost (linear in the violation magnitude) that produces a smooth, informative cost signal, making the PPO-Lagrangian update numerically well-behaved even with a sparse base reward.
3. Our Safe PPO uses a *dual-critic* architecture (one reward critic, one cost critic) and a log-parameterized λ updated by dual ascent against a cost budget d. We show empirically that the best configuration reaches the unconstrained PPO return while satisfying the cost budget.
4. We adopt a disciplined *two-stage* hyperparameter protocol (base screening, then a safety sweep on the top-three bases) and show that the base's unconstrained cost — not its return — predicts whether the constrained problem is feasible. Selecting a base by return alone reliably picks the configuration least able to meet the budget.

5. We release a clean, modular codebase with reproducible configs, automated stage runners, per-run training-process videos, and a multi-seed training script.

2 Related Work

Deep value methods. DQN [5] popularized value-based deep RL via a replay buffer and a slow-moving target network, both of which we re-use in our Phase 1 baseline.

Policy gradient methods. PPO [11] introduces a clipped surrogate objective that bounds the size of each policy update; combined with Generalized Advantage Estimation [10] it is the de-facto on-policy baseline in continuous and discrete control.

Constrained / safe RL. CMDPs [2] are the standard framework for constrained sequential decision making. CPO [1] performs a constrained policy update via a trust region. Reward-constrained Policy Optimization [13] and the PPO-Lagrangian baseline of Ray et al. [9] adapt a Lagrangian multiplier via dual ascent — the family our Safe PPO belongs to. PID-Lagrangian [12] further stabilizes the multiplier with proportional/integral/derivative control. We deliberately use the simpler dual-ascent variant because it surfaces the multiplier dynamics most clearly.

Reward shaping. Potential-based reward shaping [8] preserves the optimal policy while accelerating learning in sparse-reward problems; we use a related mechanical-energy-style shaping term throughout this paper.

3 Background

3.1 Markov Decision Processes

A discounted MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with states $\mathcal{S}$, actions $\mathcal{A}$, transition kernel P, reward R, and discount $\gamma \in [0, 1)$. The agent seeks a policy π maximizing $J_R(\pi) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)]$.

3.2 Constrained MDPs

A CMDP [2] augments an MDP with a non-negative cost $C(s, a) \geq 0$ and a budget d. The constrained problem is

$$\max_{\pi} \; J_R(\pi) \quad \text{subject to} \quad J_C(\pi) := \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t C(s_t, a_t) \right] \leq d. \tag{1}$$

The Lagrangian relaxation introduces $\lambda \geq 0$ and yields the saddle-point problem $\max_\pi \min_{\lambda \geq 0} \; J_R(\pi) - \lambda \big(J_C(\pi) - d \big)$.

3.3 PPO Clipped Surrogate Objective

For a stochastic policy π_θ with old parameters θ_{old}, PPO maximizes

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \Big[\min\Big(r_t(\theta)\, \hat{A}_t, \; \text{clip}\big(r_t(\theta), 1 - \epsilon, 1 + \epsilon\big)\, \hat{A}_t \Big) \Big], \tag{2}$$

where $r_t(\theta) = \pi_\theta(a_t \,|\, s_t) / \pi_{\theta_{\text{old}}}(a_t \,|\, s_t)$ and $\hat{A}_t$ is the GAE advantage [10]. In practice both PPO and Safe PPO add the same entropy bonus $\beta\, \mathcal{H}[\pi_\theta]$ to this objective to encourage exploration [6].

4 Method: Safe PPO with Dual-Critic and Adaptive λ

4.1 Constrained `MountainCar`

We instantiate the CMDP via two wrappers around `MountainCar-v0`: ShapedMountainCar adds a potential-style shaping bonus $\phi(s) = 1.5 \cdot |x - (-0.5)|$ and a $+100$ completion bonus on termination; ConstrainedMountainCar additionally emits a proportional safety cost

$$C(s, a) = \max\big(0, \; 100\, (|\dot{x}| - v_{\max})\big), \quad v_{\max} = 0.04. \tag{3}$$

The proportional form gives a smooth, informative signal: speeding by twice the margin yields twice the cost. We justify the specific values $v_{\max}$=0.04 and d=15 with a feasibility probe in Section 5 (Figure 1).

4.2 Dual-critic architecture

Safe PPO uses three MLP heads sharing the same input but with separate weights: an actor producing categorical logits, a reward critic $V_R(s)$, and a cost critic $V_C(s)$. For both reward and cost streams we compute Generalized Advantage Estimation [10] with discount γ=0.99 and λ_{GAE}=0.95. The reward advantage $\hat{A}_t^R$ is mean-centered and unit-scaled; the cost advantage $\hat{A}_t^C$ is only *scaled*, not centered, so that its sign continues to indicate whether an action is safer or less safe than the average. Crucially, this scaling is applied only when the batch contains non-zero cost: when no violation has occurred we set $\hat{A}_t^C$=0 rather than normalizing critic noise, so that Safe PPO reduces exactly to PPO until a genuine constraint signal appears (see Section 7).

4.3 Policy update

With clip parameter $\epsilon = 0.2$, the policy maximizes the Lagrangian-augmented surrogate

$$L(\theta) = \underbrace{\mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t^R, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t^R\big)\Big]}_{\text{reward objective}} - \lambda \cdot \underbrace{\mathbb{E}_t\Big[r_t(\theta)\,\hat{A}_t^C\Big]}_{\text{cost objective}} + \beta\,\mathcal{H}[\pi_\theta], \tag{4}$$

with entropy bonus β=0.01 [6]. The reward branch uses the PPO clip; the cost branch uses an unclipped surrogate, in line with PPO-Lagrangian baselines [9].

4.4 Adaptive Lagrangian multiplier

We parameterize $\lambda = \exp(\ell)$ with unconstrained $\ell \in \mathbb{R}$. After each policy/critic update we run one Adam step on

$$\mathcal{L}_\lambda(\ell) = -\exp(\ell)\,\big(\hat{J}_C - d\big), \tag{5}$$

where $\hat{J}_C$ is the empirical mean of the cost-return targets in the current batch. This yields the dual-ascent update $\ell \leftarrow \ell + \alpha_\lambda \exp(\ell)(\hat{J}_C - d)$, which monotonically increases λ when the constraint is violated and shrinks it otherwise. The initialization ℓ_0 and step size α_λ are tuned in Stage 2 (Section 5).

Algorithm 1 summarizes the full procedure.

Algorithm 1: Safe PPO with dual critics and adaptive λ

Initialize actor π_θ, reward critic V_R, cost critic V_C, multiplier ℓ

For epoch $k = 1 \dots K$ **do**

1. Collect T steps in ConstrainedMountainCar, storing $(s_t, a_t, r_t, c_t, V_R, V_C, \log \pi)$
2. Compute $\hat{A}^R, \hat{A}^C$ via dual GAE; targets $\hat{R}^R, \hat{R}^C$
3. Normalize $\hat{A}^R$ (mean/std); scale $\hat{A}^C$ by std only *if* batch cost >0, else $\hat{A}^C \leftarrow 0$
4. **For** $i = 1 \dots N_\pi$ **do** update θ via Adam on $-L(\theta)$ in Eq. (4)
5. **For** $i = 1 \dots N_V$ **do** update V_R, V_C via MSE on $\hat{R}^R, \hat{R}^C$
6. $\hat{J}_C \leftarrow \text{mean}(\hat{R}^C)$; $\ell \leftarrow \ell + \alpha_\lambda \exp(\ell)(\hat{J}_C - d)$

End for

5 Experimental Setup

Environment. `MountainCar-v0` [3] with maximum episode length 200 and reward −1 per step. We wrap with ShapedMountainCar for DQN/PPO and ConstrainedMountainCar ($v_{\max}$=0.04, d=15) for Safe PPO.

Constraint design. Both the speed cap $v_{\max}$ and the budget d are fixed in advance by a *feasibility probe* (Figure 1): a hand-coded speed-capped controller is swept over 200 random starts using the exact `MountainCar` physics but no learning. Two facts set the values. First, $v_{\max}$=0.04 is the tightest cap that still solves every start within the 200-step episode limit — at a cap of 0.035 the solve rate already falls to 70%, and by 0.030 the car can no longer reach the goal. Because 0.04 is also the cost threshold, any cap at or below it makes the cost identically

zero, so 0.04 is the natural floor: as slow as the agent can be while still always solving. Second, the thriftiest 100%-solving controller incurs a mean cost of ≈ 11 (worst case ≈ 16), whereas plain speeding costs ≈ 29. We set d=15 between these values — low enough to force genuinely conservative driving, yet above the achievable floor so the constraint stays feasible. The budget is thus tight but satisfiable by construction.

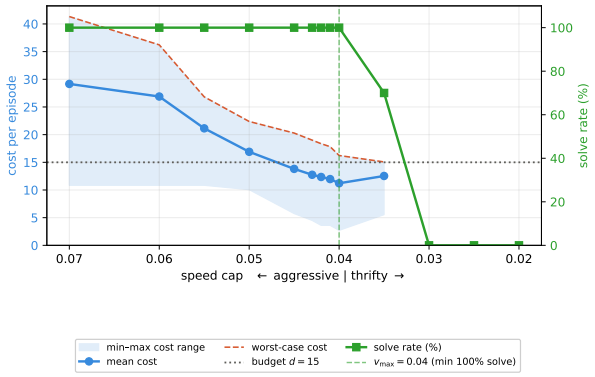

Figure 1: Feasibility probe (no learning): episode cost and solve rate versus speed cap for a hand-coded capped controller over 200 random starts. Cost falls as the cap tightens, but below $v_{\max}$=0.04 (green dashed line) the solve rate drops below 100%. The thriftiest solving cap averages cost ≈ 11, against the budget d=15 (dotted) and the ≈ 29 of unconstrained speeding, so the constraint is tight but feasible.

Networks. Two-layer MLPs with hidden size 64. DQN uses ReLU; PPO/Safe PPO use Tanh.

Optimizers. Adam everywhere. DQN: lr 10^{-3}, batch 64, target sync every five episodes, ϵ-greedy with decay 0.995 and floor 0.05. PPO / Safe PPO: V-lr 10^{-3}, 80 policy and 80 critic iterations per epoch, clip ϵ=0.2, GAE γ=0.99, λ=0.95, 4000 environment steps per epoch. The policy learning rate and entropy bonus are *tuned* in Stage 1 (below) rather than fixed.

Two-stage hyperparameter protocol. To avoid an uncontrolled joint sweep we tune two axes at a time and freeze the rest. *Stage 1* screens six unconstrained PPO bases over the grid lr $\in \{1, 3, 10\} \times 10^{-4}$ and entropy bonus $\beta \in \{0, 0.01\}$ (configs P1–P6), each over three seeds. We rank bases by the stability-aware score mean_return − std_return, keeping only those that solve (mean solved rate ≥ 0.5), and carry the top three (B1=P6, B2=P5, B3=P4) into Stage 2. We additionally log a *passive* speeding cost for each base — computed with Eq. (3) but never used in the PPO objective — which serves as a feasibility tie-breaker. *Stage 2* sweeps the safety axes $\alpha_\lambda \in \{0.02, 0.03, 0.05\}$ and $\ell_0 \in \{-1, 0\}$ on top of each base (configs S1–S18), with d=15 fixed, again over three seeds. A configuration *passes* if its final-window cost is ≤ 15 and no seed collapses (return drops $> 30\%$ from its peak without recovering within the last 20 epochs); the winner is the passing config with the highest mean return and smallest std.

Winning configuration. The selected Safe PPO (S6) uses base P6 (π-lr 10^{-3}, β=0.01) with $\alpha_\lambda = 0.05$ and $\ell_0 = 0$ ($\lambda_0 = 1$), $d = 15$.

Budgets. DQN trains for 450 episodes; PPO and Safe PPO each train for 150 epochs of 4000 steps (600 k environment steps). Stage 1 is $6 \times 3 = 18$ runs; Stage 2 is $18 \times 3 = 54$ runs.

Reproducibility. We seed Python, NumPy, and PyTorch from a single seed. The stages are launched via `python -m scripts.run_stage1 -seeds 0 1 2` and `python -m scripts.run_stage2 -seeds 0 1 2`; the latter auto-selects the top-three bases from Stage 1. All metrics are written to `results/<config>/seed_<s>/metrics.json`, and selection tables to `results/stage{1,2}_summary.md`.

Compute. On an NVIDIA RTX 5050 (8 GB) GPU each run takes ≈ 280 s. Stage 1 (18 runs) finished in **1 h 24 m** and Stage 2 (54 runs) in **4 h 12 m**, for ≈ 5.6 hours (**5 h 36 m**) of total training.

6 Results

6.1 Stage 1: base selection

Table 1 and Figure 2 report the six unconstrained PPO bases. The learning rate dominates: at 10^{-4} (P1, P2) the policy never solves within 150 epochs (0% solved, return ≈ -150); at 3×10^{-4} without entropy (P3) learning is unstable across seeds (65% solved, return -29.2 ± 92.8); and at $\{3\times10^{-4}, 10^{-3}\}$ with entropy (P4–P6) the agent solves on every seed with return ≈ 47.5. Crucially, among the three *solving* bases the passive speeding cost

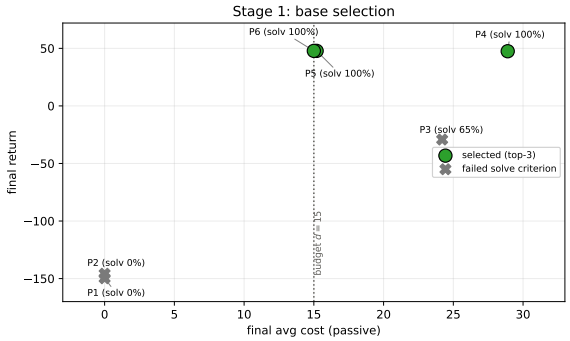

Figure 2: Stage 1 base selection: final return vs. passive cost (three-seed mean). Green circles are the three selected bases; grey crosses fail the solve criterion. The three solving bases (P4, P5, P6) share the same return but split sharply by cost, with P4 sitting nearly twice as far past the budget line as P5/P6.

differs almost two-fold: P4 reaches return 47.5 at cost 28.9, whereas P5 and P6 reach the same return at cost ≈ 15. Because return alone cannot distinguish them, this cost gap — invisible to the PPO objective — is exactly the information needed to anticipate Stage 2 feasibility.

Table 1: Stage 1 (unconstrained PPO) base screening. Final-window (last ten epochs) mean±std over three seeds. "Avg cost" is logged passively (not optimized). Score = mean − std of return; the top-three solving bases are carried into Stage 2.

ID	lr	β	Return	Avg cost	Solved	Score	Top-3
P1	10^{-4}	0	-149.7 ± 7.3	0.0	0%	−157.1	
P2	10^{-4}	0.01	-145.8 ± 0.2	0.0	0%	−146.0	
P3	3×10^{-4}	0	-29.2 ± 92.8	24.2	65%	−122.0	
P4	3×10^{-4}	0.01	47.5 ± 1.1	28.9	100%	46.4	✓
P5	10^{-3}	0	47.8 ± 1.4	15.2	100%	46.4	✓
P6	10^{-3}	0.01	47.8 ± 0.2	15.0	100%	**47.6**	✓

6.2 Stage 2: safety sweep and the role of the base

Table 2 lists all 18 constrained configurations (S1–S18) and Figure 3 plots them in the cost–return plane. The result is stark and organized entirely by base. **Every one of the six P4-based configurations fails the constraint**, clustering at cost 29–34 regardless of α_λ or ℓ_0: a base that overspeeds at cost ≈ 29 when unconstrained cannot be dragged below 15 without losing the goal. By contrast the P6- and P5-based configurations populate the feasible region, and three of them pass: S6 (P6, cost 15.0, return 47.3), S4 (P6, cost 14.7, return 46.8), and S10 (P5, cost 11.6, return 40.2 ± 8.3). This is direct confirmation that the *base's unconstrained cost*, not its return, governs constrained feasibility — and that selecting a base by return alone (which would have happily kept P4) is actively counterproductive.

A secondary effect is the initial multiplier. Across bases, $\ell_0 = 0$ ($\lambda_0 = 1$) meets the budget far more often than $\ell_0 = -1$: starting with real constraint pressure pulls cost down before the policy commits to a fast solution. The one failure mode of an aggressive start is over-constraint — S8 (P5, α_λ=0.02, ℓ_0=0) drives cost to 2.7 but collapses on one seed (-27.2 ± 52.5). The winner therefore pairs a low-cost base with a warm multiplier start and the largest sweep α_λ=0.05.

6.3 The winning policy

Figure 4 shows the winner's training dynamics. Return rises to ≈ 47 within the first ~ 40 epochs, matching the unconstrained base, and the safety cost settles right at the budget (15.0). The Lagrangian multiplier (Figure 4, right) starts at $\lambda_0 = 1$, climbs to a peak of ≈ 0.95 early in training as the agent first incurs cost, then relaxes toward ≈ 0.09 once cost is under control — exactly the self-regulating behavior expected from dual ascent. The interpretability of λ as a "how much do I weight safety?" knob is a key practical advantage of the Lagrangian approach: its trajectory reads as a running estimate of how binding the constraint currently is.

Table 2: Stage 2 (Safe PPO) sweep. Final-window (last ten epochs) mean±std over three seeds, with d=15 fixed. *Pass* requires cost ≤ 15 and no seed collapse. S6 (bold) is the winner; S8 meets the cost numerically but collapses on one seed. Bases: B1=P6, B2=P5, B3=P4.

ID	Base	α_λ	ℓ_0	Avg cost	≤ 15	Return	Pass
S1	P6	0.02	-1	19.2	no	46.0 ± 1.3	
S2	P6	0.02	0	18.9	no	29.6 ± 8.8	
S3	P6	0.03	-1	16.0	no	47.9 ± 0.8	
S4	P6	0.03	0	14.7	yes	46.8 ± 0.1	✓
S5	P6	0.05	-1	22.1	no	45.6 ± 2.8	
S6	P6	0.05	0	**15.0**	yes	$\mathbf{47.3 \pm 0.3}$	✓
S7	P5	0.02	-1	17.1	no	46.9 ± 3.5	
S8	P5	0.02	0	2.7	yes	-27.2 ± 52.5	
S9	P5	0.03	-1	17.7	no	47.8 ± 1.0	
S10	P5	0.03	0	11.6	yes	40.2 ± 8.3	✓
S11	P5	0.05	-1	17.4	no	47.5 ± 1.3	
S12	P5	0.05	0	16.3	no	48.1 ± 0.2	
S13	P4	0.02	-1	32.1	no	39.4 ± 7.5	
S14	P4	0.02	0	33.9	no	32.8 ± 12.2	
S15	P4	0.03	-1	31.6	no	36.5 ± 11.5	
S16	P4	0.03	0	29.2	no	35.4 ± 15.8	
S17	P4	0.05	-1	32.6	no	38.4 ± 9.8	
S18	P4	0.05	0	33.5	no	37.3 ± 10.3	

Table 3: Final-window performance. Mean±std across three seeds (`-seeds 0 1 2`). Final return is averaged over the last 50 episodes (DQN) or last ten epochs (PPO, Safe PPO). PPO is the selected base P6; Safe PPO is the winner S6.

Algorithm	Final return ↑	Final cost ↓	λ end	Solved
DQN	26.8 ± 2.0	–	–	99%
PPO (base P6)	$\mathbf{47.8 \pm 0.2}$	15.0	–	100%
Safe PPO (S6)	47.3 ± 0.3	$15.0 \leq 15$	0.09	100%

6.4 Comparison and summary

Figure 5 compares the three agents' shaped return as a function of environment steps. DQN reaches a steady-state return of ≈ 27; the PPO base and the constrained winner both converge to ≈ 47, with Safe PPO essentially overlapping unconstrained PPO — the constraint costs no measurable return on this task. Table 3 summarizes final-window performance.

7 Discussion

Base cost, not base return, predicts constrained feasibility. The unconstrained base determines whether the constrained problem is solvable at all. The three solving bases (P4, P5, P6) reach the same return ≈ 47.5, so a return-only rule treats them as interchangeable; yet their passive cost differs two-fold, and that gap fully determines Stage 2: every P4-based config (cost 28.9) fails the budget, while the low-cost bases P5/P6 (cost ≈ 15) yield all feasible solutions. The penalty can nudge a policy already near the budget but cannot relocate one whose fast trajectories sit deep in the violating region without sacrificing the goal. When screening bases for a downstream constraint, log the passive cost and prefer low-cost bases among those that solve; selecting by return alone picks the worst starting point.

Sparse reward delays constraint pressure. Because `MountainCar` returns -1 per step until termination, the agent has little incentive to act until reward shaping takes hold. Until it regularly reaches the flag it also rarely hits the speed limit, so the cost critic and multiplier receive almost no signal; only once reward and cost signals co-activate does dual ascent begin useful work.

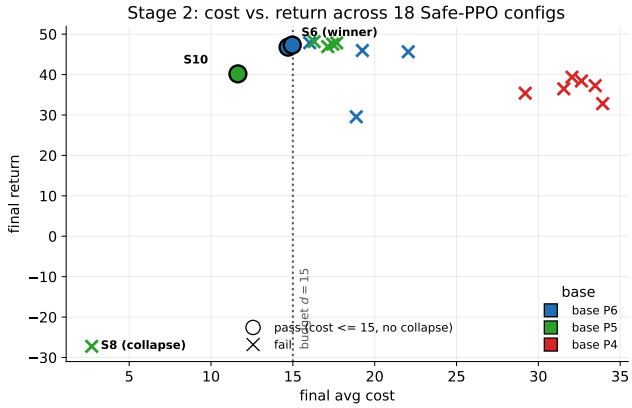

Figure 3: Stage 2: final cost vs. return for all 18 Safe-PPO configurations (three-seed mean), colored by base (P6 blue, P5 green, P4 red). Filled circles pass (cost ≤ 15, no collapse); crosses fail. All P4-based configs sit far past the budget; the feasible passing configs are built on the low-cost bases P6 and P5. S8 meets the cost numerically but collapses.

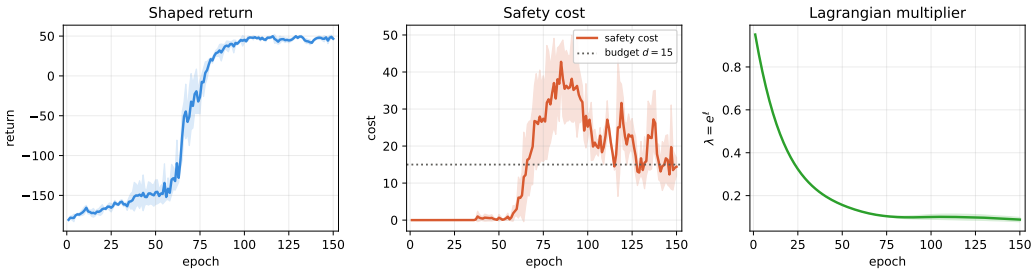

Figure 4: Winning Safe PPO (S6) dynamics, three-seed mean±std: shaped return (left), safety cost against the budget d=15 (middle), and the Lagrangian multiplier λ (right). The constraint becomes binding early; λ peaks then relaxes as cost converges toward the budget with no return loss.

Cost-advantage normalization in the zero-cost regime. One implementation detail proved decisive. While the environment emits no cost, the cost advantage $\hat{A}^C$ is pure cost-critic noise; standardizing it to unit variance rescales that noise to the reward-advantage magnitude, so the $\lambda\hat{A}^C$ term injects $\mathcal{O}(\lambda)$ gradient noise on *every* update. Guarding the normalization so that $\hat{A}^C$=0 whenever the batch has no cost — letting Safe PPO reduce exactly to PPO until a genuine violation occurs — restores PPO-level discovery while leaving constrained behavior unchanged. We recommend this guard for any PPO-Lagrangian method on sparse-cost CMDPs.

The initial multiplier is a feasibility knob, with a collapse failure mode. Stage 2 reveals a non-monotone role for ℓ_0. A small start ($\ell_0 = -1$) lets the agent commit to a fast solution before the penalty bites, and most such configs end above the budget; a warm start ($\ell_0 = 0$, $\lambda_0 = 1$) applies real pressure early and meets the budget far more often — the winner uses it. But pushing harder is not free: the over-constrained S8 (low α_λ with the warm start) drives cost to 2.7 yet collapses the policy on one seed. The multiplier dynamics thus have a usable but narrow operating band, which PID-Lagrangian [12] widens by damping the response automatically.

Soft budget, met in practice. The tuned winner converges to cost 15.0, exactly meeting d=15, at no return cost. Because the penalty is *soft* (proportional, never strictly infeasible), this reflects a genuine equilibrium between reward and cost gradients rather than a hard projection. Feasibility is nonetheless thin: hand-coded speed-capped controllers bottom out near cost 11, so d=15 sits just above the achievable minimum. A hard-constraint variant (e.g., CPO [1]) would instead project onto the feasible set; we leave that to future work.

Limitations. (i) We screen with three seeds; the winner is stable (47.3 ± 0.3), but several Stage 2 configs show high across-seed variance, so it should be re-run with five seeds before being treated as final — our selection script flags this. (ii) The multiplier is updated against the *discounted* cost-return estimate while feasibility is checked against the *undiscounted* episode cost; reconciling the two is a worthwhile refinement. (iii) `MountainCar`

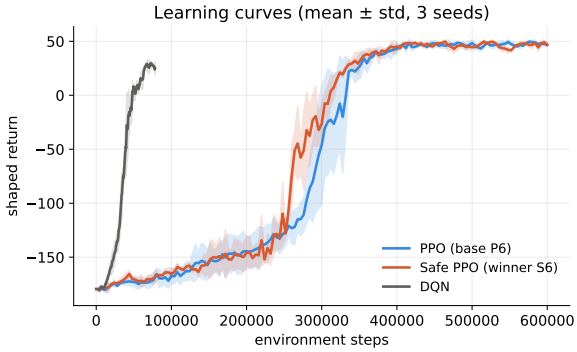

Figure 5: Shaped return vs. environment steps for DQN, the PPO base (P6), and the Safe PPO winner (S6). Mean±std across three seeds. The two PPO variants are nearly indistinguishable.

is a simple two-dimensional environment; conclusions for higher-dimensional or visual settings require further experiments (e.g., 9).

8 Conclusion

We presented a progressively constructed comparison of DQN, PPO, and Safe PPO on the `MountainCar-v0` sparse-reward benchmark, organized as a two-stage hyperparameter study. The tuned Safe PPO — a dual-critic actor-critic with a log-parameterized multiplier and a proportional soft cost — meets the budget ($15.0 \leq 15$) at a return indistinguishable from the unconstrained base. Our central finding is that constrained feasibility is governed by the base policy's unconstrained cost, not its return: low-cost bases yield all feasible solutions, while a high-cost base of equal return fails in every safety configuration. The simple setting keeps the dual-ascent dynamics visible and pedagogically clear. Future work includes a five-seed confirmation, a PID-Lagrangian variant, hard-constraint comparisons, and higher-dimensional CMDP benchmarks.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning (ICML)*, 2017.

[2] Eitan Altman. *Constrained Markov Decision Processes*. CRC Press, 1999.

[3] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[4] Yinlam Chow, Mohammad Ghavamzadeh, Lucas Janson, and Marco Pavone. Risk-constrained reinforcement learning with percentile risk criteria. *Journal of Machine Learning Research*, 18(167), 2018.

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518 (7540), 2015.

[6] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

[7] Andrew William Moore. Efficient memory-based learning for robot control. *PhD thesis, University of Cambridge*, 1990.

[8] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. *International Conference on Machine Learning (ICML)*, 99, 1999.

[9] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. Technical report, OpenAI, 2019. URL `https://cdn.openai.com/safexp-short.pdf`.

[10] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[12] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by pid lagrangian methods. *International Conference on Machine Learning (ICML)*, 2020.

[13] Chen Tessler, Daniel J Mankowitz, and Shie Mannor. Reward constrained policy optimization. *International Conference on Learning Representations (ICLR)*, 2019.